

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

		PRÁCTICA DE LABORATORIO	
CARRERA: Computación		ASIGNATURA: Computación Paralela	
NRO. PRÁCTICA:	5	TÍTULO PRÁCTICA: Desarrollo e implementación de aplicaciones de computo paralelo basado en paso de mensajes	
OBJETIVOS ALCANZADO: Diseñar, implementar con mpi4py y evaluar una solución paralela basada en paso de mensajes sobre un clúster, analizando el impacto de la distribución de tareas, la comunicación y la sincronización en el rendimiento general frente a su versión secuencial.			
REPOSITORIO: https://github.com/KarenS22/Simulador-Ambiental.git (rama: mpi)			
ACTIVIDADES DESARROLLADAS			
1. Configuración de la Red y Credenciales SSH: Se enlazaron físicamente los 3 nodos en una red local común con ssh. Se generaron llaves SSH en la máquina coordinadora (cthu lhu) y se exportaron las llaves públicas a los esclavos slave1 y slave2 para que MPICH orquestase la ejecución remota de procesos sin solicitar contraseña. <pre> > ssh slave1 pyenv: no such command `virtualenv-init' [siryorch@karen-22 ~]\$ exit cerrar sesión Connection to slave1 closed.  ~  > ssh slave2 pyenv: no such command `virtualenv-init' [siryorch@archlinux ~]\$ _ </pre>			
2. Montaje del Sistema de Archivos Compartido (Samba & CIFS): Se configuró un servidor Samba en la PC Principal compartiendo el directorio del proyecto. En las máquinas esclavas (slave1 y slave2) se instalaron los paquetes de cliente cifs-utils y se montó la carpeta en una ruta idéntica del sistema (/home/flamenco/home/). Esto garantiza que el clúster ejecute físicamente el mismo código fuente. <pre> /home/flamenco/home > P npt 147 http /home/flamenco/home > P npt 147 71 sudo mount -t cifs -o guest,uid=\$(id -u),gid=\$(id -g),dir_mode=0777,file_mode=0777 //172.26.211.198/Compartido /home/flamenco/home </pre>			

3. Desarrollo del Algoritmo del Coordinador (ControladorMPI): Se codificó la distribución de carga y asignación estática de las estaciones. Se implementó un reparto cíclico usando la operación modular $r = i \% \text{size}$. Si el resultado de la asignación es 0, la estación es procesada por el propio proceso maestro (Rank 0), optimizando el uso de la CPU. Para cualquier otro residuo r , la estación es asignada a los procesos trabajadores.

```
# core/mmpi/controlador.py
for i, est in enumerate(self.estaciones):
    r = i % size
    if r == 0:
        local_assignments.append(est)
    else:
        worker_assignments[r].append(est)
```

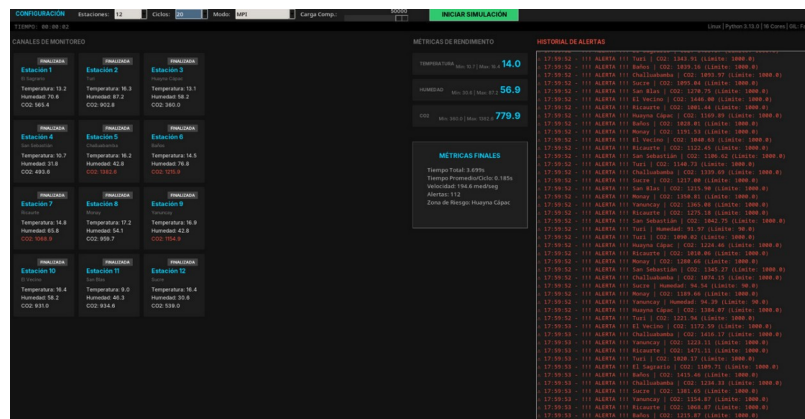
4. Implementación de los Procesos Trabajadores (worker.py): Se programó la lógica para bloquear las máquinas esclavas esperando la señal de inicio "start". Al iniciar, los trabajadores obtienen su identificador de PID (os.getpid()) y nombre de host físico (socket.gethostname()) y los envían inicialmente al master. Luego, ejecutan de forma paralela e independiente la simulación ambiental y pesado cómputo matemático de sus estaciones correspondientes.

```
# core/mmpi/worker.py
pid = os.getpid()
hostname = socket.gethostname()
# Notificar a la UI sobre la información de este proceso (PID, Rank y Host)
for est in estaciones:
    comm.send(("info_proceso", (est.id_estacion, pid, rank, display_host)),
              dest=0)
```

5. Configuración del Entorno de Sincronización Simultánea (comm.Barrier): Se incorporó un bloqueo de barrera (comm.Barrier()) al final de cada ciclo de simulación tanto para el maestro como para los esclavos. Esto sincroniza las lecturas temporales de la simulación urbana de manera idéntica en toda la red y evita desbordes o inconsistencias de datos consolidados.

```
# Sincronización al final del ciclo por barrera en red
comm.Barrier()
```

6. Mapeo de Control e Integración con la Interfaz Tkinter: Se actualizó la GUI en ventana.py con el modo de ejecución MPI.



	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

RESULTADO(S) OBTENIDO(S):

Se realizaron pruebas exhaustivas de la aplicación sobre el clúster variando la cantidad de procesos MPI lanzados con la flag -n de mpiexec. La versión con $N=1$ actúa como el modo de ejecución secuencial local de referencia (punto de base).

1. Pruebas Variando N (Carga Computacional = 10,000 | Ciclos = 10)

Número de Procesos (N)	4 Estaciones	8 Estaciones	12 Estaciones
N = 1	0.493s	1.000s	1.565s
N = 4	0.361s	0.564s	0.739s
N = 8	0.313s	0.476s	0.516s
N = 12	0.512s	0.580s	0.559s

- A medida que aumentamos el número de trabajadores en la red, el tiempo se reduce notablemente. Para 12 estaciones el tiempo pasa de 1.565s a 0.516s alcanzando una aceleración de 3.03x.
- Al incrementar a 12 procesos, el tiempo de simulación comienza a degradarse ligeramente (subiendo de 0.316s a 0.559s para 12 estaciones). Esto ocurre debido a que la sobrecarga del paso de mensajes de TCP/IP, SSH de MPICH y latencias del sistema de compartición Samba superan el beneficio de división de cómputo en volumen de datos pequeños.

2. Pruebas de Carga Extrema (Clúster con $N=12$ fijo | 12 Estaciones | Carga Computacional = 50,000)

Ciclos de Simulación	Tiempo Paralelo Distribuido
10 Ciclos	1.746s
20 Ciclos	3.699s
30 Ciclos	5.828s

Al incrementar la carga computacional base de las estaciones a 50,000 iteraciones matemáticas, el sistema paralelo MPI escala linealmente con los ciclos de ejecución debido a la mayor proporción de tiempo dedicada al procesamiento de CPU pura frente al costo relativo de comunicación por red.

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

3. Comparativa Global de Rendimiento y Velocidad (12 Estaciones | 10 Ciclos | Carga = 10,000)

Para esta comparativa se calculó el rendimiento global en base a las 360 mediciones totales generadas (12 estaciones x 3 variables x 10 ciclos):

Paradigma de Ejecución	Tiempo de Ejecución (T)	Velocidad de Cómputo	Aceleramiento (S) vs Secuencial	Eficiencia Relativa (E)
Secuencial (MPI N = 1)	1.565s	230.0 med/seg	1.00x (Base)	100.0%
Hilos (Threading)	0.583s	617.5 med/seg	2.68x	-
Procesos (Multiprocessing)	0.650s	553.8 med/seg	2.40x	-
MPI (Clúster Físico, N = 8)	0.516s	697.7 med/seg	3.03x	37.8% (sobre 8 procs)

La solución distribuida MPI con 8 procesos alcanza la velocidad de cómputo más alta con 697.7 mediciones procesadas por segundo, superando a los hilos y procesos locales gracias a que la carga de cálculo de CPU intensa se divide físicamente en tres computadoras reales (c_{thulu}, s_{lave1}, s_{lave2}).

Aunque MPI demuestra la mayor velocidad absoluta en mediciones/segundo, la eficiencia es mayor en los hilos locales a nivel de un único procesador físico al no lidiar con retardo de red.

CONCLUSIONES:

El modelo de paso de mensajes de MPI con mpi4py permite escalar una aplicación por encima de la cantidad de núcleos físicos de una sola computadora, dividiendo la carga entre múltiples máquinas.

El rendimiento de la red (latencia y ancho de banda) y la sobrecarga de envío influyen directamente en la escalabilidad de procesos, observándose que agregar más nodos (como en $N=12$) puede saturar el bus y aumentar el tiempo de respuesta.

RECOMENDACIONES:

Utilizar un número de procesos (N) equilibrado con la cantidad de núcleos de CPU y estaciones simuladas, evitando saturar la interfaz de red del nodo master (c_{thulu}).

ANÁLISIS COMPARATIVO:

1. ¿Cómo se distribuyen los datos y tareas entre los procesos trabajadores en el modo MPI?

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

El Coordinador (Rank 0) implementa un reparto dinámico y cíclico de las estaciones usando la operación módulo $r = i \% \text{size}$. Si la división resulta en 0, la estación es procesada por el hilo principal del Coordinador. Para cualquier valor $r > 0$, la estación se despacha vía `comm.send` al proceso trabajador con ese Rank en el clúster. Así, cada estación es administrada por un subproceso físico independiente en red, el cual cuenta con su propio PID de sistema operativo, su Rank MPI y su HostName mapeado en la interfaz.

2. ¿Cómo se comunican los procesos y cómo se consolidan los resultados en el nodo master en el entorno distribuido?

La comunicación es híbrida:

- Punto a punto: Los nodos trabajadores envían inicialmente metadatos descriptivos ("info_proceso" con PID, Rank y Hostname) y luego, durante el ciclo, mensajes individuales de "estado" y "medicion" usando `comm.send()`. El orquestador los recibe de forma asíncrona con `comm.recv(source=MPI.ANY_SOURCE)`.
- Colectiva: Para sincronizar las iteraciones en cada paso de ciclo, se invoca `comm.Barrier()`. Esto frena la sobrecarga de envío asíncrono y asegura que el clúster avance coordinadamente. El nodo master consolida los datos agregándolos a su historial global e invocando a la GUI.

3. ¿Qué impacto tiene la latencia de red en una solución distribuida (MPI) frente a la escalabilidad de procesos?

A menor volumen de cálculo (como carga = 10,000), la latencia de red tiene un impacto dominante. Como se evidencia al pasar de $N=8$ a $N=12$, agregar más procesos distribuidos no mejora indefinidamente la velocidad. Esto demuestra la Ley de Amdahl modificada para entornos distribuidos: la comunicación de red impone un límite de velocidad que solo se reduce cuando la carga computacional individual de cada trabajador aumenta lo suficiente (como en la prueba de carga 50,000).

4. ¿Qué impacto tiene la barrera colectiva (Barrier()) en el rendimiento general bajo memoria distribuida?

La barrera garantiza la sincronización estricta por ciclo temporal (ideal para simular la toma de datos coordinada de una ciudad). Sin embargo, introduce un problema de "cuello de botella de balance de carga": la velocidad total del clúster se ve limitada por la computadora más lenta. El nodo que finalice primero sus cálculos deberá quedarse inactivo en `comm.Barrier()` esperando a que los demás completen su tarea y alcancen el mismo punto.

5. ¿Qué dificultades reales se encontraron durante el despliegue del clúster físico y cómo se solucionaron?

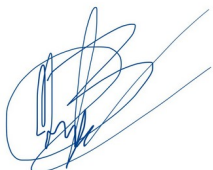
Incompatibilidad de entornos Python: La ejecución fallaba inicialmente debido a diferencias leves de versiones de Python entre los nodos. Se resolvió homogenizando las versiones de los sistemas Arch/Manjaro.

Envío remoto con SSH: MPICH solicitaba contraseña por cada proceso lanzado en red. Se solucionó configurando un par de claves SSH públicas/privadas y añadiéndolas a `authorized_keys`.

Acceso mutuo al código de ejecución: Mantener copias locales del script provocaba que cualquier cambio desalineara la ejecución. Se solventó montando un directorio común en red mediante Samba y `cifs-utils` en la ruta `/home/flamenco/home/`.

Problema con la descarga de MPICH: En uno de los nodos la descarga e instalación automática del binario falló repetidamente. Esto obligó a realizar una compilación manual de la biblioteca desde las fuentes originales (`tar.gz`), configurando e instalando manualmente con `./configure`, `make` y `sudo make install`.

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

Nombre de estudiante:	Firma de estudiante:
Mateo Barzallo	
Jorge Cueva	
Karen Quito	